

THE WARRIOR'S

DOCUMENTATION ARSENAL

Complete Gear Checklist · Databricks · Apache Spark · Django

192	1022	11	8
TOTAL TASKS	TOTAL XP	SECTIONS	RANKS

WARRIOR RANKS — YOUR PROGRESSION							
Raw Recruit	Squire	Scout	Soldier	Knight	Warlord	Champion	Data Legend
Start	10 tasks	25 tasks	45 tasks	65 tasks	85 tasks	110 tasks	All 192

TABLE OF CONTENTS — FULL ARSENAL

[SHIELD] Databricks — Lakehouse Platform

17 tasks · 65 xp

- [] What is the Databricks Lakehouse Platform?
- [] Lakehouse vs data warehouse vs data lake
- [] Control plane vs data plane architecture
- [] Databricks Runtime versions
- [] All-purpose clusters — configuration
- [] Job clusters — ephemeral compute
- [] Cluster modes: single node, standard, high concurrency
- [] Auto-scaling and auto-termination
- [] Cluster policies
- [] Notebooks: cell types and magic commands (%sql %python %run)
- [] Notebook widgets — parameterisation
- [] Notebook-scoped libraries
- [] Databricks Repos — Git integration
- [] Branching, committing, and pulling in Repos
- [] Databricks SQL and SQL warehouses overview
- [] Serverless vs classic SQL warehouses
- [] Connecting BI tools to Databricks SQL

[SWORD] Delta Lake — Your Primary Weapon

20 tasks · 116 xp

- [] What is Delta Lake?
- [] The _delta_log transaction log
- [] Managed vs external Delta tables
- [] CREATE TABLE USING DELTA
- [] DESCRIBE DETAIL and DESCRIBE HISTORY
- [] ALTER TABLE — adding and renaming columns
- [] Schema enforcement and schema evolution
- [] MERGE INTO — upserts *** MOST TESTED ***
- [] UPDATE and DELETE on Delta tables
- [] Slowly Changing Dimensions with MERGE (SCD1 and SCD2)
- [] Time travel: VERSION AS OF
- [] Time travel: TIMESTAMP AS OF
- [] RESTORE TABLE to previous version
- [] VACUUM — removing old file versions
- [] OPTIMIZE — file compaction
- [] Z-ORDER BY — multi-dimensional clustering
- [] Data skipping with column statistics
- [] ANALYZE TABLE — update statistics
- [] Table properties and delta options
- [] SHALLOW and DEEP CLONE Delta tables

[SPEAR] Structured Streaming and Auto Loader

15 tasks · 92 xp

- [] Structured Streaming fundamentals overview
- [] Streaming sources: Delta, Kafka, cloud files
- [] Streaming sinks: Delta, memory, console
- [] Output modes: append, complete, update
- [] Trigger modes: once, continuous, availableNow
- [] Checkpointing and fault tolerance
- [] Watermarking for late-arriving data
- [] Reading Delta table as a streaming source
- [] Writing streams to Delta table sinks
- [] foreachBatch for custom streaming logic
- [] Auto Loader overview — cloudFiles format
- [] Auto Loader: schema inference and hints
- [] Auto Loader: rescued data column for malformed records
- [] Auto Loader vs COPY INTO — when to use each
- [] COPY INTO — idempotent batch file ingestion

[FLAME BANNER] Delta Live Tables (DLT)

7 tasks · 38 xp

- ☐ What is Delta Live Tables? — declarative pipelines
- ☐ DLT: streaming tables vs materialised view tables
- ☐ DLT expectations: WARN, DROP, FAIL
- ☐ DLT pipeline: development vs production mode
- ☐ Triggered vs continuous DLT pipeline execution
- ☐ Monitoring DLT pipelines with event logs
- ☐ DLT with Unity Catalog — governed pipelines

[BOOTS] Spark SQL — Mobility and Speed

23 tasks · 134 xp

- ☐ SparkSession — the entry point to Spark
- ☐ SELECT, FROM, WHERE, LIMIT basics
- ☐ GROUP BY, HAVING, ORDER BY
- ☐ INNER JOIN, LEFT JOIN, RIGHT JOIN
- ☐ FULL OUTER JOIN, CROSS JOIN, self-join
- ☐ Semi-join (LEFT SEMI) and anti-join (LEFT ANTI)
- ☐ UNION, INTERSECT, EXCEPT set operations
- ☐ Subqueries and correlated subqueries
- ☐ CTEs — WITH clause
- ☐ Window functions: RANK, DENSE_RANK, ROW_NUMBER
- ☐ Window functions: LAG, LEAD, NTILE
- ☐ Window functions: running SUM and AVG OVER partitions
- ☐ PIVOT and UNPIVOT
- ☐ CASE WHEN, IF conditional expressions
- ☐ String functions: regexp_replace, split, trim, concat
- ☐ Date and timestamp functions
- ☐ EXPLODE and FLATTEN — array expansion
- ☐ Higher-order functions: TRANSFORM, FILTER, AGGREGATE
- ☐ Struct and Map type operations
- ☐ from_json and to_json — JSON parsing
- ☐ NULL handling: COALESCE, NULLIF, IS NULL, NVL
- ☐ SQL UDFs — CREATE FUNCTION in Databricks
- ☐ Broadcast hints and query optimisation hints

[GAUNTLETS] PySpark DataFrame API — Striking Power

26 tasks · 138 xp

- ☐ DataFrame creation: spark.read.csv, json, parquet
- ☐ spark.createDataFrame from list or pandas DataFrame
- ☐ Schema definition with StructType and StructField
- ☐ select() and selectExpr() transformations
- ☐ filter() and where() — row filtering
- ☐ withColumn() and withColumnRenamed()
- ☐ drop() and dropDuplicates()
- ☐ groupBy() and agg() with multiple aggregations
- ☐ join() with all join type strings
- ☐ orderBy() and sort() with ascending/descending
- ☐ limit() and sample() — data reduction
- ☐ union() and unionByName()
- ☐ na.fill(), na.drop(), na.replace() — null handling
- ☐ Actions: show(), count(), collect(), first(), take()
- ☐ write.format().mode().save() — writing output
- ☐ createOrReplaceTempView() and createGlobalTempView()
- ☐ Method chaining — building readable transformation chains
- ☐ Python UDFs: udf() and @udf decorator
- ☐ Pandas UDFs (vectorised) with @pandas_udf
- ☐ explain() — reading Spark query execution plans
- ☐ cache() and persist() — memory optimisation
- ☐ Broadcast variables and accumulators
- ☐ Spark MLlib Pipeline API — chaining stages
- ☐ MLlib: transformers vs estimators distinction
- ☐ MLlib: classification and regression models

[HELMET] Databricks Workflows — Command and Strategy

15 tasks · 72 xp

- [] Jobs overview — what is a Databricks job?
- [] Creating a job with notebook, Python, or SQL task
- [] Job task types: notebook, Python script, JAR, SQL, DLT
- [] Multi-task workflows and task dependencies
- [] Passing parameters between tasks using task values
- [] Cron scheduling syntax
- [] File-arrival triggers
- [] Concurrent run policies: allow, skip, wait
- [] Retry policies for failed tasks
- [] Email alerts on job success and failure
- [] Viewing job run history and output logs
- [] Databricks CLI setup and authentication
- [] CLI commands: jobs run now, clusters list
- [] REST API — trigger jobs programmatically
- [] Secrets management for pipeline credentials

[CLOAK] Unity Catalog — Stealth and Governance

13 tasks · 60 xp

- [] Unity Catalog overview and benefits
- [] Three-level namespace: catalog.schema.table
- [] Metastore vs legacy Hive metastore
- [] CREATE CATALOG and CREATE SCHEMA commands
- [] GRANT and REVOKE on catalogs, schemas, tables
- [] Privilege types: SELECT, MODIFY, CREATE, USE CATALOG
- [] Service principals vs user accounts
- [] Data owner and data steward role distinction
- [] Automatic column-level data lineage
- [] Audit logs for data access events
- [] Row filters using dynamic views
- [] Column masks for PII data protection
- [] Tags and data discovery in Unity Catalog

[MEDALLION] Medallion Architecture — Sacred Code

6 tasks · 36 xp

- [] Medallion architecture philosophy and overview
- [] Bronze layer — raw ingestion design patterns
- [] Silver layer — validation, cleansing, deduplication
- [] Gold layer — business-level aggregates and KPIs
- [] Connecting layers with DLT pipeline declarations
- [] Data quality enforcement contracts per layer

[SPELL TOME I] Django — Models and Database ORM

18 tasks · 100 xp

- [] Django overview — MTV architecture pattern
- [] Project structure: manage.py, settings, apps
- [] Models — field types: CharField, IntegerField, DateField
- [] Relationships: ForeignKey, ManyToManyField, OneToOneField
- [] Meta class — ordering, verbose names, indexes, constraints
- [] Migrations: makemigrations and migrate commands
- [] Data migrations — RunPython and custom operations
- [] QuerySet API: filter(), exclude(), get(), all()
- [] QuerySet chaining and lazy evaluation behaviour
- [] annotate(), aggregate(), F objects, Q objects
- [] select_related() — forward FK joins in one query
- [] prefetch_related() — reverse and M2M relations
- [] values() and values_list() — dict and tuple output
- [] order_by(), distinct(), count(), exists() methods
- [] Raw SQL: raw() and connection.execute()
- [] Custom model managers and QuerySet methods
- [] Model signals: post_save, pre_delete, post_migrate

[] Database transactions and atomic() context manager

[SPELL TOME II] Django — Views, URLs and HTTP Layer

15 tasks · 71 xp

- [] Function-based views — request in, response out
- [] Class-based views — reusable, extensible view classes
- [] Generic CBVs: ListView, DetailView, CreateView, UpdateView
- [] URL configuration with path() and re_path()
- [] URL namespaces and reverse() for URL generation
- [] Middleware — global request and response processing
- [] HttpRequest and HttpResponse objects in detail
- [] Django forms: Form class, validation, clean methods
- [] Sessions and cookies management
- [] Caching framework: per-view, per-site, low-level cache
- [] Authentication: login, logout, login_required, permissions
- [] Custom user model with AbstractUser and AbstractBaseUser
- [] Django admin — auto-generated management interface
- [] Static files and media files serving in development
- [] Testing in Django — TestCase, Client, assertContains

[SPELL TOME III] Django REST Framework — ML Delivery API

17 tasks · 100 xp

- [] DRF overview, installation, and project setup
- [] Serializers — ModelSerializer and base Serializer class
- [] SerializerMethodField and custom field validation
- [] APIView — base class handling GET, POST, PUT, DELETE
- [] ViewSets and Routers — automatic endpoint generation
- [] Generic views: ListCreateAPIView, RetrieveUpdateDestroyAPIView
- [] Authentication: Token, Session, JWT
- [] Permissions: IsAuthenticated, IsAdminUser, custom classes
- [] Throttling and rate limiting for API protection
- [] Filtering, searching, and ordering via query params
- [] Pagination: PageNumberPagination, CursorPagination
- [] Nested serializers and writable relational fields
- [] Serving a scikit-learn ML model via DRF endpoint
- [] API versioning: URL, header, and namespace strategies
- [] Django deployment with Gunicorn and Nginx
- [] Environment variables and settings for production
- [] Dockerising a Django and ML application together



[SHIELD] Databricks — Lakehouse Platform

Your body armour. Every concept here protects your data architecture from collapse.

Databricks Data Engineering Docs | docs.databricks.com/aws/en/data-engineering/

17 tasks 65
xp

☐	What is the Databricks Lakehouse Platform? <i>Iron Breastplate — foundational concept of the entire exam</i> docs.databricks.com/aws/en/introduction/	+5xp
☐	Lakehouse vs data warehouse vs data lake <i>Chest plate — know the battlefield before you fight</i> docs.databricks.com/aws/en/lakehouse/	+5xp
☐	Control plane vs data plane architecture <i>Spine guard — who controls what in the system</i> docs.databricks.com/aws/en/getting-started/overview.html	+4xp
☐	Databricks Runtime versions <i>Armour alloy — choose the right material for the job</i> docs.databricks.com/aws/en/release-notes/runtime/	+3xp
☐	All-purpose clusters — configuration <i>Standard shield — general combat cluster</i> docs.databricks.com/aws/en/compute/configure.html	+4xp
☐	Job clusters — ephemeral compute <i>Battle shield — lightweight, auto-terminates after battle</i> docs.databricks.com/aws/en/compute/configure.html	+4xp
☐	Cluster modes: single node, standard, high concurrency <i>Shield variants — pick your defence posture</i> docs.databricks.com/aws/en/compute/cluster-config-best-practices.html	+4xp
☐	Auto-scaling and auto-termination <i>Shield auto-repair — never waste gold on idle armour</i> docs.databricks.com/aws/en/compute/configure.html	+4xp
☐	Cluster policies <i>Shield governance — enforce rules across the army</i> docs.databricks.com/aws/en/compute/cluster-policies.html	+3xp
☐	Notebooks: cell types and magic commands (%sql %python %run) <i>Shield emblem — command the notebook battlefield</i> docs.databricks.com/aws/en/notebooks/	+5xp
☐	Notebook widgets — parameterisation <i>Shield handle — add input controls to your notebook</i> docs.databricks.com/aws/en/notebooks/widgets.html	+3xp
☐	Notebook-scoped libraries <i>Shield polish — install libraries per session</i> docs.databricks.com/aws/en/libraries/notebook-libraries.html	+3xp
☐	Databricks Repos — Git integration <i>Shield lineage — version control your battle plans</i> docs.databricks.com/aws/en/repos/	+4xp
☐	Branching, committing, and pulling in Repos <i>Shield reinforcement — collaborate without overwriting</i> docs.databricks.com/aws/en/repos/git-operations-with-repos.html	+4xp
☐	Databricks SQL and SQL warehouses overview <i>Shield signal — query data for business commanders</i> docs.databricks.com/aws/en/sql/	+4xp
☐	Serverless vs classic SQL warehouses <i>Shield tier — cost vs speed tradeoff decision</i> docs.databricks.com/aws/en/compute/sql-warehouse/create.html	+3xp
☐	Connecting BI tools to Databricks SQL <i>Shield API port — plug Power BI and Tableau in</i> docs.databricks.com/aws/en/integrations/bi/	+3xp

WEAPON FORGED · 17 tasks complete · +65 XP



[SWORD] Delta Lake — Your Primary Weapon

Your main offensive weapon. Master every strike — Delta is 22% of the DEA exam.

Databricks Delta Lake Docs | docs.databricks.com/aws/en/delta/

20 tasks 116
xp

☐	What is Delta Lake? <i>Sword forged — ACID on Parquet, the foundation weapon</i> docs.databricks.com/aws/en/delta/	+6xp
☐	The <code>_delta_log</code> transaction log <i>Sword spine — every commit is recorded here</i> docs.databricks.com/aws/en/delta/transaction-log.html	+6xp
☐	Managed vs external Delta tables <i>Blade type A vs B — know which you fully control</i> docs.databricks.com/aws/en/delta/managed-vs-unmanaged.html	+5xp
☐	CREATE TABLE USING DELTA <i>First forge strike — create your weapon</i> docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-ddl-create-table-using.html	+5xp
☐	DESCRIBE DETAIL and DESCRIBE HISTORY <i>Blade inspection — examine your weapon status</i> docs.databricks.com/aws/en/delta/history.html	+5xp
☐	ALTER TABLE — adding and renaming columns <i>Blade reshaping — evolve schema without rewriting</i> docs.databricks.com/aws/en/delta/column-mapping.html	+5xp
☐	Schema enforcement and schema evolution <i>Blade hardening — control what data enters your table</i> docs.databricks.com/aws/en/delta/schema-evolution.html	+6xp
☐	MERGE INTO — upserts *** MOST TESTED *** <i>DIAMOND EDGE — ultimate strike: insert+update+delete in one</i> docs.databricks.com/aws/en/sql/language-manual/delta-merge-into.html	+10xp
☐	UPDATE and DELETE on Delta tables <i>Surgical strikes — precision modifications</i> docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-dml-update.html	+6xp
☐	Slowly Changing Dimensions with MERGE (SCD1 and SCD2) <i>Advanced combo — maintain full historical records</i> docs.databricks.com/aws/en/delta/merge.html	+7xp
☐	Time travel: VERSION AS OF <i>Temporal strike — attack with historical data</i> docs.databricks.com/aws/en/delta/history.html	+8xp
☐	Time travel: TIMESTAMP AS OF <i>Time anchor — pinpoint an exact historical moment</i> docs.databricks.com/aws/en/delta/history.html	+7xp
☐	RESTORE TABLE to previous version <i>Blade restoration — undo battle damage completely</i> docs.databricks.com/aws/en/sql/language-manual/delta-restore.html	+6xp
☐	VACUUM — removing old file versions <i>Blade cleaning — remove obsolete data versions safely</i> docs.databricks.com/aws/en/sql/language-manual/delta-vacuum.html	+5xp
☐	OPTIMIZE — file compaction <i>Blade honing — compact small files into powerful strikes</i> docs.databricks.com/aws/en/sql/language-manual/delta-optimize.html	+6xp
☐	Z-ORDER BY — multi-dimensional clustering <i>Blade focus — co-locate related data for faster reads</i> docs.databricks.com/aws/en/delta/data-skipping.html	+6xp
☐	Data skipping with column statistics <i>Feint technique — skip irrelevant data blocks entirely</i> docs.databricks.com/aws/en/delta/data-skipping.html	+5xp
☐	ANALYZE TABLE — update statistics <i>Battle intelligence — keep optimizer informed</i> docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-dml-analyze-table.html	+4xp

☐	Table properties and delta options <i>Blade enchantments — configure table behaviours</i> docs.databricks.com/aws/en/delta/table-properties.html	+4xp
☐	SHALLOW and DEEP CLONE Delta tables <i>Mirror blade — duplicate without duplicating cost</i> docs.databricks.com/aws/en/delta/clone.html	+4xp

WEAPON FORGED · 20 tasks complete · +116 XP



[SPEAR] Structured Streaming and Auto Loader

Your ranged weapon. Strike data before it lands. Real-time is your long-range attack.

15 tasks 92 xp

[Databricks Structured Streaming Docs](#) | docs.databricks.com/aws/en/structured-streaming/

☐	Structured Streaming fundamentals overview <i>Spear shaft forged — foundation of real-time combat</i> docs.databricks.com/aws/en/structured-streaming/	+6xp
☐	Streaming sources: Delta, Kafka, cloud files <i>Spear targets — where your stream originates</i> docs.databricks.com/aws/en/structured-streaming/data-sources.html	+5xp
☐	Streaming sinks: Delta, memory, console <i>Spear landing zones — where your stream terminates</i> docs.databricks.com/aws/en/structured-streaming/data-sources.html	+5xp
☐	Output modes: append, complete, update <i>Strike modes — how data materialises at the target</i> docs.databricks.com/aws/en/structured-streaming/output-modes.html	+7xp
☐	Trigger modes: once, continuous, availableNow <i>Strike speed — control your throw frequency</i> docs.databricks.com/aws/en/structured-streaming/triggers.html	+7xp
☐	Checkpointing and fault tolerance <i>Spear lanyard — never lose a throw, always recover</i> docs.databricks.com/aws/en/structured-streaming/query-recovery.html	+7xp
☐	Watermarking for late-arriving data <i>Spear range limiter — handle stragglers gracefully</i> docs.databricks.com/aws/en/structured-streaming/watermarks.html	+6xp
☐	Reading Delta table as a streaming source <i>Infinite quiver — Delta feeds the stream endlessly</i> docs.databricks.com/aws/en/structured-streaming/delta-streaming.html	+6xp
☐	Writing streams to Delta table sinks <i>Strike landing — persist stream results durably</i> docs.databricks.com/aws/en/structured-streaming/delta-streaming.html	+6xp
☐	foreachBatch for custom streaming logic <i>Custom spear tip — apply any transformation mid-flight</i> docs.databricks.com/aws/en/structured-streaming/foreach.html	+5xp
☐	Auto Loader overview — cloudFiles format <i>Auto-loading crossbow — monitor cloud storage automatically</i> docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/	+8xp
☐	Auto Loader: schema inference and hints <i>Arrow shape detection — auto-detect incoming data schema</i> docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/schema.html	+6xp
☐	Auto Loader: rescued data column for malformed records <i>Broken arrow recovery — capture bad records safely</i> docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/rescue.html	+5xp
☐	Auto Loader vs COPY INTO — when to use each <i>Weapon selection — crossbow vs cannon, know the difference</i> docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/	+7xp
☐	COPY INTO — idempotent batch file ingestion <i>Siege cannon — load files once, safely, idempotently</i> docs.databricks.com/aws/en/sql/language-manual/delta-copy-into.html	+6xp



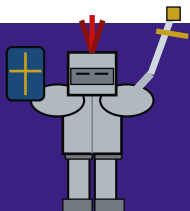
[FLAME BANNER] Delta Live Tables (DLT)

Your battle standard. DLT pipelines self-heal, enforce quality, and lead your data army.

7 tasks 38 xp

Databricks Delta Live Tables Docs | docs.databricks.com/aws/en/delta-live-tables/

- ☐ **What is Delta Live Tables? — declarative pipelines** +6xp
 Banner unfurled — DLT pipeline philosophy
docs.databricks.com/aws/en/delta-live-tables/
- ☐ **DLT: streaming tables vs materialised view tables** +6xp
 Banner variants — live vs saved snapshot
docs.databricks.com/aws/en/delta-live-tables/tables.html
- ☐ **DLT expectations: WARN, DROP, FAIL** +7xp
 Banner quality laws — enforce or punish bad data
docs.databricks.com/aws/en/delta-live-tables/expectations.html
- ☐ **DLT pipeline: development vs production mode** +5xp
 Training ground vs battlefield — deliberate mode switching
docs.databricks.com/aws/en/delta-live-tables/updates.html
- ☐ **Triggered vs continuous DLT pipeline execution** +5xp
 Burst charge vs sustained assault — pick your tempo
docs.databricks.com/aws/en/delta-live-tables/updates.html
- ☐ **Monitoring DLT pipelines with event logs** +5xp
 Battle debrief — read the pipeline event log carefully
docs.databricks.com/aws/en/delta-live-tables/observability.html
- ☐ **DLT with Unity Catalog — governed pipelines** +4xp
 Banner and cloak combined — lineage-tracked pipelines
docs.databricks.com/aws/en/delta-live-tables/unity-catalog.html



[BOOTS] Spark SQL — Mobility and Speed

Speed across any terrain. Spark SQL lets you move through distributed data at full scale.

23 tasks 134 xp

Apache Spark SQL + PySpark Docs | spark.apache.org/docs/latest/api/python/reference/pyspark.sql/

- ☐ **SparkSession — the entry point to Spark** +5xp
 Boots buckled — `spark.sql()` and `spark.read()` mastered
spark.apache.org/docs/latest/api/python/reference/pyspark.sql/spark_session.html
- ☐ **SELECT, FROM, WHERE, LIMIT basics** +4xp
 Basic step — every data journey starts here
spark.apache.org/docs/latest/sql-ref.html
- ☐ **GROUP BY, HAVING, ORDER BY** +5xp
 Formation march — aggregate and sort your forces
spark.apache.org/docs/latest/sql-ref.html
- ☐ **INNER JOIN, LEFT JOIN, RIGHT JOIN** +6xp
 Force merging — combine armies from two fronts
spark.apache.org/docs/latest/sql-ref.html
- ☐ **FULL OUTER JOIN, CROSS JOIN, self-join** +5xp
 Total war — every possible force combination
spark.apache.org/docs/latest/sql-ref.html

☐	Semi-join (LEFT SEMI) and anti-join (LEFT ANTI) <i>Spy tactics — filter by what exists or not elsewhere</i> spark.apache.org/docs/latest/sql-ref.html	+6xp
☐	UNION, INTERSECT, EXCEPT set operations <i>Army merging laws — combine or subtract result sets</i> spark.apache.org/docs/latest/sql-ref.html	+5xp
☐	Subqueries and correlated subqueries <i>Tactical manoeuvre — query inside a query</i> spark.apache.org/docs/latest/sql-ref.html	+6xp
☐	CTEs — WITH clause <i>Agility rune — name your sub-battles for clarity</i> spark.apache.org/docs/latest/sql-ref.html	+6xp
☐	Window functions: RANK, DENSE_RANK, ROW_NUMBER <i>Speed enchantment I — rank your soldiers precisely</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/window.html	+8xp
☐	Window functions: LAG, LEAD, NTILE <i>Speed enchantment II — look forward and backward in data</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/window.html	+8xp
☐	Window functions: running SUM and AVG OVER partitions <i>Speed enchantment III — running totals across groups</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/window.html	+7xp
☐	PIVOT and UNPIVOT <i>Formation flip — rotate rows to columns and back</i> spark.apache.org/docs/latest/sql-ref.html	+6xp
☐	CASE WHEN, IF conditional expressions <i>Conditional strike — branch your battle logic</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/conditional.html	+5xp
☐	String functions: regexp_replace, split, trim, concat <i>Blade precision — cut and shape text data cleanly</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/string.html	+5xp
☐	Date and timestamp functions <i>Temporal mastery — parse, format, and diff dates</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/datetime.html	+5xp
☐	EXPLODE and FLATTEN — array expansion <i>Parkour boots I — expand nested arrays into rows</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/collection.html	+7xp
☐	Higher-order functions: TRANSFORM, FILTER, AGGREGATE <i>Parkour boots II — apply logic inside arrays in-place</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/collection.html	+7xp
☐	Struct and Map type operations <i>Terrain navigation — work with complex nested types</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/collection.html	+6xp
☐	from_json and to_json — JSON parsing <i>Cipher boots — decode and encode JSON payloads</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/misc.html	+6xp
☐	NULL handling: COALESCE, NULLIF, IS NULL, NVL <i>Gap-filling soles — handle missing data terrain safely</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/conditional.html	+5xp
☐	SQL UDFs — CREATE FUNCTION in Databricks <i>Custom boot mould — define your own SQL functions</i> docs.databricks.com/aws/en/udf/	+6xp
☐	Broadcast hints and query optimisation hints <i>Boots afterburner — guide the optimizer manually</i> spark.apache.org/docs/latest/sql-performance-tuning.html	+5xp

WEAPON FORGED · 23 tasks complete · +134 XP



[GAUNTLETS] PySpark DataFrame API — Striking Power

Your bare-hand combat. Code you write yourself — no AI crutch in interviews.

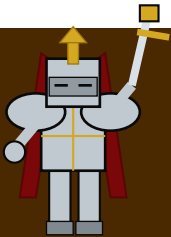
26 tasks 138
xp

PySpark DataFrame API Reference |
spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html

☐	DataFrame creation: <code>spark.read.csv</code>, <code>json</code>, <code>parquet</code>	+5xp
	<i>Gauntlet base — pick up raw data with your hands</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	<code>spark.createDataFrame</code> from list or pandas DataFrame	+4xp
	<i>Forge data from scratch — create test datasets</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/spark_session.html	
☐	Schema definition with <code>StructType</code> and <code>StructField</code>	+5xp
	<i>Gauntlet shaping — define exact data structure upfront</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/types.html	
☐	<code>select()</code> and <code>selectExpr()</code> transformations	+5xp
	<i>First strike — pick and compute exactly the columns needed</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	<code>filter()</code> and <code>where()</code> — row filtering	+5xp
	<i>Defensive parry — keep only the data rows you want</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	<code>withColumn()</code> and <code>withColumnRenamed()</code>	+5xp
	<i>Knuckle reshape — add new columns and rename existing</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	<code>drop()</code> and <code>dropDuplicates()</code>	+4xp
	<i>Clean strike — remove columns and duplicate rows</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	<code>groupBy()</code> and <code>agg()</code> with multiple aggregations	+6xp
	<i>Power grip — group and aggregate with full force</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	<code>join()</code> with all join type strings	+6xp
	<i>Two-handed slam — combine two DataFrames precisely</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	<code>orderBy()</code> and <code>sort()</code> with ascending/descending	+4xp
	<i>Formation order — rank your results correctly</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	<code>limit()</code> and <code>sample()</code> — data reduction	+3xp
	<i>Controlled swing — cap or randomly sample your data</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	<code>union()</code> and <code>unionByName()</code>	+4xp
	<i>Force combination — stack two DataFrames vertically</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	<code>na.fill()</code>, <code>na.drop()</code>, <code>na.replace()</code> — null handling	+5xp
	<i>Wound treatment — handle nulls cleanly in your data</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	Actions: <code>show()</code>, <code>count()</code>, <code>collect()</code>, <code>first()</code>, <code>take()</code>	+5xp
	<i>Strike delivery — trigger actual Spark computation</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	<code>write.format().mode().save()</code> — writing output	+5xp
	<i>Store your spoils — write results to cloud storage</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	<code>createOrReplaceTempView()</code> and <code>createGlobalTempView()</code>	+5xp
	<i>Bridge building — switch between DataFrame API and SQL</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	Method chaining — building readable transformation chains	+5xp
	<i>Combo strikes — chain 5+ transformations smoothly</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	
☐	Python UDFs: <code>udf()</code> and <code>@udf</code> decorator	+7xp
	<i>Gauntlet spike I — add custom Python logic to DataFrames</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/udf.html	

☐	Pandas UDFs (vectorised) with @pandas_udf <i>Gauntlet spike II — 10x faster custom functions</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/udf.html	+7xp
☐	explain() — reading Spark query execution plans <i>X-ray vision — see exactly how Spark executes your code</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	+6xp
☐	cache() and persist() — memory optimisation <i>Gauntlet charge — hold data in memory for repeated use</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html	+5xp
☐	Broadcast variables and accumulators <i>War supplies — share data and track counts across cluster</i> spark.apache.org/docs/latest/api/python/reference/pyspark.sql/	+5xp
☐	Spark MLlib Pipeline API — chaining stages <i>Magic gauntlet — chain ML stages like DataFrame transforms</i> spark.apache.org/docs/latest/api/python/reference/pyspark.ml/	+7xp
☐	MLlib: transformers vs estimators distinction <i>Spell components — the two stage types in every ML pipeline</i> spark.apache.org/docs/latest/api/python/reference/pyspark.ml/	+6xp
☐	MLlib: classification and regression models <i>Combat spells — RandomForest, LogisticRegression at scale</i> spark.apache.org/docs/latest/api/python/reference/pyspark.ml/classification.html	+7xp
☐	MLlib: feature engineering — StringIndexer, VectorAssembler, Scaler <i>Spell ingredients — prepare features for the model</i> spark.apache.org/docs/latest/api/python/reference/pyspark.ml/feature.html	+7xp

WEAPON FORGED · 26 tasks complete · +138 XP



[HELMET] Databricks Workflows — Command and Strategy

Your strategic mind protection. Orchestrate armies of tasks without losing a soldier.

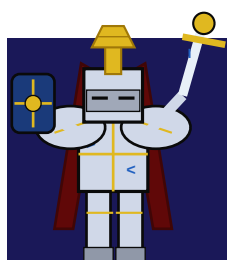
Databricks Workflows and Jobs Docs | docs.databricks.com/aws/en/jobs/

15 tasks 72
xp

☐	Jobs overview — what is a Databricks job? <i>Helmet base — the command structure explained clearly</i> docs.databricks.com/aws/en/jobs/	+4xp
☐	Creating a job with notebook, Python, or SQL task <i>First command — dispatch troops to battle</i> docs.databricks.com/aws/en/jobs/create-run-jobs.html	+4xp
☐	Job task types: notebook, Python script, JAR, SQL, DLT <i>Helmet variants — command different soldier types</i> docs.databricks.com/aws/en/jobs/	+5xp
☐	Multi-task workflows and task dependencies <i>Tactical chain of command — sequential and parallel tasks</i> docs.databricks.com/aws/en/jobs/create-run-jobs.html	+6xp
☐	Passing parameters between tasks using task values <i>Command signals — share intelligence between tasks</i> docs.databricks.com/aws/en/jobs/share-task-context.html	+6xp
☐	Cron scheduling syntax <i>Battle clock — trigger attacks on exact schedule</i> docs.databricks.com/aws/en/jobs/schedule-jobs.html	+5xp
☐	File-arrival triggers <i>Sentry trigger — auto-launch when data arrives</i> docs.databricks.com/aws/en/jobs/schedule-jobs.html	+5xp
☐	Concurrent run policies: allow, skip, wait <i>Queue discipline — manage overlapping battles</i> docs.databricks.com/aws/en/jobs/create-run-jobs.html	+4xp

☐	Retry policies for failed tasks <i>Battle revival — configure automatic retries</i> docs.databricks.com/aws/en/jobs/create-run-jobs.html	+5xp
☐	Email alerts on job success and failure <i>War messenger — notify commanders of all outcomes</i> docs.databricks.com/aws/en/jobs/create-run-jobs.html	+4xp
☐	Viewing job run history and output logs <i>After-action report — examine every past battle</i> docs.databricks.com/aws/en/jobs/create-run-jobs.html	+4xp
☐	Databricks CLI setup and authentication <i>Radio headset — command the army from terminal</i> docs.databricks.com/aws/en/dev-tools/cli/	+5xp
☐	CLI commands: jobs run now, clusters list <i>Voice commands — trigger and inspect via command line</i> docs.databricks.com/aws/en/dev-tools/cli/	+5xp
☐	REST API — trigger jobs programmatically <i>Remote command centre — automate from any system</i> docs.databricks.com/aws/en/dev-tools/api/	+5xp
☐	Secrets management for pipeline credentials <i>Cipher helmet — store passwords safely, never in plaintext</i> docs.databricks.com/aws/en/security/secrets/	+5xp

WEAPON FORGED · 15 tasks complete · +72 XP



[CLOAK] Unity Catalog — Stealth and Governance

Controls who sees what. The cloak of invisibility and access control.

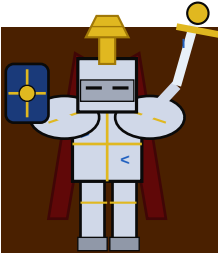
Databricks Unity Catalog Docs | docs.databricks.com/aws/en/data-governance/unity-catalog/

13 tasks 60
xp

☐	Unity Catalog overview and benefits <i>Cloak woven — centralised governance across workspaces</i> docs.databricks.com/aws/en/data-governance/unity-catalog/	+4xp
☐	Three-level namespace: catalog.schema.table <i>Cloak layers — master the full hierarchy (most tested in D5)</i> docs.databricks.com/aws/en/data-governance/unity-catalog/	+6xp
☐	Metastore vs legacy Hive metastore <i>Old cloak vs new — understand the upgrade path</i> docs.databricks.com/aws/en/data-governance/unity-catalog/	+4xp
☐	CREATE CATALOG and CREATE SCHEMA commands <i>Cloak territory — define your data kingdom zones</i> docs.databricks.com/aws/en/data-governance/unity-catalog/create-catalogs.html	+5xp
☐	GRANT and REVOKE on catalogs, schemas, tables <i>Cloak seal — decide who enters which zone</i> docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/	+6xp
☐	Privilege types: SELECT, MODIFY, CREATE, USE CATALOG <i>Access levels — different keys for different doors</i> docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/privileges.html	+5xp
☐	Service principals vs user accounts <i>Spy vs soldier — machine identity vs human identity</i> docs.databricks.com/aws/en/administration-guide/users-groups/	+4xp
☐	Data owner and data steward role distinction <i>Kingdom hierarchy — know the chain of authority</i> docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/	+4xp
☐	Automatic column-level data lineage <i>Cloak tracker — see every data footprint automatically</i> docs.databricks.com/aws/en/data-governance/unity-catalog/data-lineage.html	+5xp

- ☐ **Audit logs for data access events** +4xp
Spy journal — log every door that was opened
docs.databricks.com/aws/en/administration-guide/account-settings/audit-logs.html
- ☐ **Row filters using dynamic views** +5xp
Cloak patch I — hide specific rows from enemies
docs.databricks.com/aws/en/data-governance/unity-catalog/row-and-column-filters.html
- ☐ **Column masks for PII data protection** +5xp
Cloak patch II — mask sensitive columns automatically
docs.databricks.com/aws/en/data-governance/unity-catalog/row-and-column-filters.html
- ☐ **Tags and data discovery in Unity Catalog** +3xp
Kingdom map — label and find your data assets
docs.databricks.com/aws/en/data-governance/unity-catalog/tags.html

WEAPON FORGED · 13 tasks complete · +60 XP



[MEDALLION] Medallion Architecture — Sacred Code

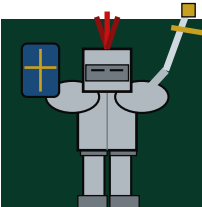
Bronze, Silver, Gold — the three sacred laws every clean pipeline must obey.

6 tasks 36 xp

Databricks Medallion Architecture Guide | docs.databricks.com/aws/en/lakehouse/medallion.html

- ☐ **Medallion architecture philosophy and overview** +5xp
Sacred code revealed — the three-layer data philosophy
docs.databricks.com/aws/en/lakehouse/medallion.html
- ☐ **Bronze layer — raw ingestion design patterns** +6xp
Bronze earned — store everything raw, lose absolutely nothing
docs.databricks.com/aws/en/lakehouse/medallion.html
- ☐ **Silver layer — validation, cleansing, deduplication** +7xp
Silver earned — purify, deduplicate, enforce quality contracts
docs.databricks.com/aws/en/lakehouse/medallion.html
- ☐ **Gold layer — business-level aggregates and KPIs** +7xp
Gold earned — serve only what decisions require
docs.databricks.com/aws/en/lakehouse/medallion.html
- ☐ **Connecting layers with DLT pipeline declarations** +6xp
Sacred chain — wire Bronze to Silver to Gold with DLT
docs.databricks.com/aws/en/lakehouse/medallion.html
- ☐ **Data quality enforcement contracts per layer** +5xp
Quality vows — what each layer must guarantee downstream
docs.databricks.com/aws/en/lakehouse/medallion.html

WEAPON FORGED · 6 tasks complete · +36 XP



[SPELL TOME I] Django — Models and Database ORM

Spellbook Part I. Command databases with Python incantations.

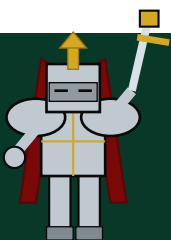
18 tasks 100 xp

Django Documentation — Models and Database | docs.djangoproject.com/en/5.2/topics/db/

- ☐ **Django overview — MTV architecture pattern** +4xp
Tome cover — Model, Template, View: the three pillars
docs.djangoproject.com/en/5.2/intro/overview/
- ☐ **Project structure: manage.py, settings, apps** +4xp
Spell preparation — understand the kingdom layout
docs.djangoproject.com/en/5.2/intro/tutorial01/

☐	Models — field types: CharField, IntegerField, DateField <i>Spell page 1 — define the shape of your data universe</i> docs.djangoproject.com/en/5.2/topics/db/models/	+6xp
☐	Relationships: ForeignKey, ManyToManyField, OneToOneField <i>Relational runes — link your data kingdoms together</i> docs.djangoproject.com/en/5.2/topics/db/models/#relationships	+6xp
☐	Meta class — ordering, verbose names, indexes, constraints <i>Tome meta — control model behaviour and performance</i> docs.djangoproject.com/en/5.2/ref/models/options/	+4xp
☐	Migrations: makemigrations and migrate commands <i>Spell evolution — upgrade your schema without data loss</i> docs.djangoproject.com/en/5.2/topics/migrations/	+6xp
☐	Data migrations — RunPython and custom operations <i>Advanced casting — write your own schema transformation spells</i> docs.djangoproject.com/en/5.2/topics/migrations/#data-migrations	+5xp
☐	QuerySet API: filter(), exclude(), get(), all() <i>Summon spell — call forth exactly the data you need</i> docs.djangoproject.com/en/5.2/ref/models/querysets/	+7xp
☐	QuerySet chaining and lazy evaluation behaviour <i>Spell delay — QuerySets are lazy until materialised</i> docs.djangoproject.com/en/5.2/ref/models/querysets/#when-querysets-are-evaluated	+6xp
☐	annotate(), aggregate(), F objects, Q objects <i>Calculation runes — compute derived values on the fly</i> docs.djangoproject.com/en/5.2/topics/db/aggregation/	+7xp
☐	select_related() — forward FK joins in one query <i>Efficiency rune I — eliminate the N+1 query curse</i> docs.djangoproject.com/en/5.2/ref/models/querysets/#select-related	+7xp
☐	prefetch_related() — reverse and M2M relations <i>Efficiency rune II — batch-fetch related records smartly</i> docs.djangoproject.com/en/5.2/ref/models/querysets/#prefetch-related	+7xp
☐	values() and values_list() — dict and tuple output <i>Spell simplification — return flat data not full objects</i> docs.djangoproject.com/en/5.2/ref/models/querysets/#values	+5xp
☐	order_by(), distinct(), count(), exists() methods <i>Utility spells — sort, deduplicate, count, check existence</i> docs.djangoproject.com/en/5.2/ref/models/querysets/	+5xp
☐	Raw SQL: raw() and connection.execute() <i>Ancient tongue — drop to raw SQL when ORM is insufficient</i> docs.djangoproject.com/en/5.2/topics/db/sql/	+5xp
☐	Custom model managers and QuerySet methods <i>Spell specialisation — add domain-specific query methods</i> docs.djangoproject.com/en/5.2/topics/db/managers/	+5xp
☐	Model signals: post_save, pre_delete, post_migrate <i>Spell triggers — react automatically to data lifecycle events</i> docs.djangoproject.com/en/5.2/topics/signals/	+5xp
☐	Database transactions and atomic() context manager <i>Spell isolation — wrap operations in ACID transaction safety</i> docs.djangoproject.com/en/5.2/topics/db/transactions/	+6xp

WEAPON FORGED · 18 tasks complete · +100 XP



[SPELL TOME II] Django — Views, URLs and HTTP Layer

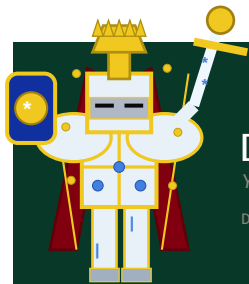
Spellbook Part II. Route requests to the right incantation every time.

Django Documentation — HTTP Layer | docs.djangoproject.com/en/5.2/topics/http/

15 tasks 71
xp

☐	Function-based views — request in, response out <i>Simple cast — the most direct spell form</i> docs.djangoproject.com/en/5.2/topics/http/views/	+5xp
☐	Class-based views — reusable, extensible view classes <i>Spell class hierarchy — DRY view patterns</i> docs.djangoproject.com/en/5.2/topics/class-based-views/	+6xp
☐	Generic CBVs: ListView, DetailView, CreateView, UpdateView <i>Spell shortcuts — pre-built views for common CRUD patterns</i> docs.djangoproject.com/en/5.2/ref/class-based-views/	+5xp
☐	URL configuration with path() and re_path() <i>Spell routing table — map URLs to views cleanly</i> docs.djangoproject.com/en/5.2/topics/http/urls/	+5xp
☐	URL namespaces and reverse() for URL generation <i>Return path rune — never hardcode a URL again</i> docs.djangoproject.com/en/5.2/topics/http/urls/#naming-url-patterns	+4xp
☐	Middleware — global request and response processing <i>Spell interceptors — transform every request globally</i> docs.djangoproject.com/en/5.2/topics/http/middleware/	+5xp
☐	HttpRequest and HttpResponse objects in detail <i>Spell vessels — the carrier of all web messages</i> docs.djangoproject.com/en/5.2/ref/request-response/	+4xp
☐	Django forms: Form class, validation, clean methods <i>Input shield — validate data before it enters the kingdom</i> docs.djangoproject.com/en/5.2/topics/forms/	+5xp
☐	Sessions and cookies management <i>Spell memory — remember the warrior between visits</i> docs.djangoproject.com/en/5.2/topics/http/sessions/	+4xp
☐	Caching framework: per-view, per-site, low-level cache <i>Spell memory cache — avoid repeating heavy computations</i> docs.djangoproject.com/en/5.2/topics/cache/	+5xp
☐	Authentication: login, logout, login_required, permissions <i>Castle gates — control who enters your data kingdom</i> docs.djangoproject.com/en/5.2/topics/auth/	+6xp
☐	Custom user model with AbstractUser and AbstractBaseUser <i>Identity rune — define your own warrior identity model</i> docs.djangoproject.com/en/5.2/topics/auth/customizing/	+5xp
☐	Django admin — auto-generated management interface <i>Commander panel — manage your realm without extra code</i> docs.djangoproject.com/en/5.2/ref/contrib/admin/	+4xp
☐	Static files and media files serving in development <i>Supply depot — serve CSS, JS, images during development</i> docs.djangoproject.com/en/5.2/howto/static-files/	+3xp
☐	Testing in Django — TestCase, Client, assertContains <i>War games — test your spells safely before production</i> docs.djangoproject.com/en/5.2/topics/testing/	+5xp

WEAPON FORGED · 15 tasks complete · +71 XP



[SPELL TOME III] Django REST Framework — ML Delivery API

Your most powerful tome. DRF forges APIs that serve ML models to the world.

[Django REST Framework Documentation | djangorestframework.org/](https://djangorestframework.org/)

17 tasks 100
xp

☐	DRF overview, installation, and project setup <i>Tome III unlocked — the weapon for ML model delivery</i> djangorestframework.org/	+5xp
--------------------------	--	------

☐	Serializers — ModelSerializer and base Serializer class <i>Data translator — convert models to JSON and back cleanly</i> django-rest-framework.org/api-guide/serializers/	+7xp
☐	SerializerMethodField and custom field validation <i>Advanced translation — add computed fields and rules</i> django-rest-framework.org/api-guide/serializers/#serializermethodfield	+6xp
☐	APIView — base class handling GET, POST, PUT, DELETE <i>Spell base class — handle all HTTP verbs manually</i> django-rest-framework.org/api-guide/views/	+6xp
☐	ViewSet and Routers — automatic endpoint generation <i>Auto-routing rune — generate all CRUD endpoints from one class</i> django-rest-framework.org/api-guide/viewsets/	+6xp
☐	Generic views: ListCreateAPIView, RetrieveUpdateDestroyAPIView <i>Power shortcuts — full CRUD API in minimal code</i> django-rest-framework.org/api-guide/generic-views/	+6xp
☐	Authentication: Token, Session, JWT <i>API gate — control who can call your ML endpoints</i> django-rest-framework.org/api-guide/authentication/	+7xp
☐	Permissions: IsAuthenticated, IsAdminUser, custom classes <i>Spell permission system — who can do what at each endpoint</i> django-rest-framework.org/api-guide/permissions/	+6xp
☐	Throttling and rate limiting for API protection <i>Spell cooldown — protect your ML API from overload</i> django-rest-framework.org/api-guide/throttling/	+5xp
☐	Filtering, searching, and ordering via query params <i>API refinement — let callers filter your ML outputs</i> django-rest-framework.org/api-guide/filtering/	+5xp
☐	Pagination: PageNumberPagination, CursorPagination <i>Result management — deliver data in controlled chunks</i> django-rest-framework.org/api-guide/pagination/	+4xp
☐	Nested serializers and writable relational fields <i>Deep spell structure — nested objects in one response</i> django-rest-framework.org/api-guide/relations/	+5xp
☐	Serving a scikit-learn ML model via DRF endpoint <i>ULTIMATE SPELL — load model, predict, return JSON response</i> django-rest-framework.org/tutorial/quickstart/	+10xp
☐	API versioning: URL, header, and namespace strategies <i>Spell versioning — v1 and v2 without breaking callers</i> django-rest-framework.org/api-guide/versioning/	+4xp
☐	Django deployment with Gunicorn and Nginx <i>Release ritual — unleash your API in production server</i> docs.djangoproject.com/en/5.2/howto/deployment/	+6xp
☐	Environment variables and settings for production <i>Spell secrets — never hardcode passwords in spellbooks</i> docs.djangoproject.com/en/5.2/topics/settings/	+5xp
☐	Dockerising a Django and ML application together <i>Armour containerisation — ship your entire arsenal in a box</i> docs.djangoproject.com/en/5.2/howto/deployment/	+7xp

WEAPON FORGED · 17 tasks complete · +100 XP